

Architecture et qualite du code

BruitTrack - revue technique de la conception logicielle

Date	23 aout 2026
Portee	src/bruittrack, tests, packaging, scripts d'installation et CI. Les regles metrologiques et la securite reseau ne sont traitees que lorsqu'elles revelent un defaut de conception ou de code.
Base examinee	Revision Git 600a0be, sans modification du code produit. Le dossier non suivi files/ est hors portee.
Avis global	Architecture en couches legere et adaptee au produit. La qualite est bonne sur le coeur DSP et la persistance, mais trois problemes de robustesse et de responsabilites doivent etre corriges avant de qualifier le daemon 24/7.

Conclusion

Le projet a les bons axes de modularisation : configuration typée, capture remplaçable, DSP deterministe, detection separée, stockage transactionnel et serveur de visualisation autonome. La suite de 92 tests passe et le lint passe. La lisibilité du coeur est satisfaisante pour un projet de cette taille.

Les zones les plus fragiles sont l'orchestration runtime, les erreurs I/O et la presentation : l'alerte de capture lente est contournée dans le chemin Engine.start, certains echecs sont avales sans contrat observable, et viz.py comme __main__.py cumulent trop de responsabilites. Le rapport propose une trajectoire de refactorisation incrementale, sans introduire de dependance lourde.

1. Methode et indicateurs

La revue est fondee sur lecture des sources, analyse statique AST, inspection des relations d'import, des contrats d'erreur et des tests. Les observations sont des constats de code, references avec chemin et ligne de la revision auditee.

Indicateurs observes

Indicateur	Valeur	Lecture
Taille coeur Python	2 984 lignes, 9 modules	Taille raisonnable; trois fichiers depassent 400 lignes.
Modules les plus denses	viz.py 543, store.py 462, __main__.py 430, events.py 412 lignes	Priorite de fractionnement : UI, CLI puis detection.
Fonctions les plus longues	EventDetector.update 105, load_config 100, cmd_test 89, main 83 lignes	La longueur reste abordable, mais plusieurs decisions sont melees dans une meme methode.
Tests	92 passes en 13,62 s	Bon socle rapide et deterministe; pas de mesure de couverture ni de recette hardware CI.
Lint	ruff check passe; format check echoue sur 2 fichiers	Hygiene presque bonne, mais gate CI actuellement rouge sur ce point.

Limites de la revue

- Pas d'instrumentation sur le HP T620 ni de carte audio disponible : la robustesse temps reel est evaluee par le code et les tests simules, pas re-certifiee en situation.
- Les chiffres de taille sont des comptages de lignes et de fonctions, non une mesure de complexite cyclomatique formelle.
- Les modifications non suivies du depot n'ont pas ete examinees ni alterees.

2. Architecture actuelle

Les dependances suivent pour l'essentiel le flux unidirectionnel attendu. config est le noyau de parametres; capture, dsp et legal sont des services de bas niveau; events compose le domaine; store isole SQLite; pipeline orchestre; viz et __main__ restent les adaptateurs d'entree/sortie.

Couche	Responsabilite	Evaluation
Configuration	Dataclasses et chargement TOML dans config.py.	Bonne centralisation. load_config concentre parsing, coercion, resolution de chemins et validation.
Acquisition	AudioCapture et MockAudioCapture dans capture.py.	Bonne abstraction testable et import sounddevice differe; protocole formel absent.
Traitement	Filtre, decimation, Welch, floor et correlation dans dsp.py.	Cohesion technique elevee, etat borne et calculs isolables.
Domaine	Detection, fingerprint, cluster et evenement dans events.py.	Modele clair; machine a etats et politique de classification dans une seule classe de 220 lignes.
Persistence	Lots, SQLite, triage et retention dans store.py.	Bonne encapsulation; API de lecture renvoie des dicts non typés et agrege aussi le domaine cluster.
Adaptateurs	CLI dans __main__.py et HTTP/HTML dans viz.py.	Fonctionnels mais surcharges : presentation, orchestration et transformation de donnees sont entremeles.

Forces architecturales

- Les objets injectables capture et store permettent les tests sans materiel et sans base disque.
- Le code hot path reste dans numpy/SciPy, avec flux et buffers bornes : c'est coherent avec le budget du T620.
- La base est derriere EventStore, ce qui limite les requetes SQL dispersees. Les connexions disque courtes facilitent la coexistence capture et visualization.
- La CLI n'impose pas sounddevice a l'import, ce qui preserve le diagnostic et les tests hors materiel.

3. Constats prioritaires de qualite de code

ID	Severite	Constat	Justification
A-01	HAUTE	Le controle de sante capture est ignore par Engine.start.	pipeline.py:173 lit le bloc, puis appelle step(raw_block). Or step n'appelle _check_capture_health qu'au branchement raw_block is None (113-118). Les blocs lents reels ne declenchent donc pas l'alerte promise.
A-02	HAUTE	Des erreurs I/O sont masquees et certains contrats deviennent mensongers.	events.py:401-412 avale toute exception d'ecriture d'exemplaire mais conserve le flag exemplaire. store.py:230-236 conserve un lot mais ne remonte pas l'echec a Engine. capture.py:171-176 masque toute erreur d'arret.
A-03	MOYENNE	UI et CLI concentrent trop de responsabilites.	viz.py contient une tres grande chaine HTML/JS et le handler HTTP; __main__.py melange parseur, metriques /proc, formatage, lecture WAV, subprocess et orchestration. La testabilite par composant diminue.
A-04	MOYENNE	ClusterIndex est une recherche lineaire sans index secondaire.	events.py:172-186 compare chaque fingerprint a chaque cluster. La reconstruction charge jusqu'a 100 000 clusters; le cout de detection augmente lineairement avec l'historique.
A-05	MOYENNE	Contrats de type et frontieres de couche trop vagues.	Any est utilise dans capture, viz et CLI; on_tick est dict[str, Any]; EventStore renvoie list[dict[str, Any]]. Cela reporte des erreurs de contrat vers l'execution.
A-06	BASSE	Hygiene formelle incomplete.	ruff format --check signale tests/test_viz_api.py et tools/_purge_0hz.py; quelques docstrings/commentaires affichent des caracteres mal encodes.

4. Analyse detaillee des constats

A-01 - Observabilite contournée dans le chemin nominal

Le design cherche a avertir apres trois lectures lentes. Ce comportement est encapsule dans `_check_capture_health`, ce qui est sain. Mais `Engine.start` possede sa propre lecture de capture et transmet ensuite le bloc a `step`; cette forme d'appel saute le crochet de sante. Le testable `unit step(raw_block)` et l'execution production ne suivent donc pas exactement le meme chemin.

Correction ciblée : appeler `_check_capture_health` immediatement apres `get_block` dans `Engine.start`, ou deleguer toute lecture a `step`. Ajouter un test de boucle `Engine.start` avec une capture lente simulée et assertion de `log/compteur`.

A-02 - Erreurs silencieuses et etat non fiable

Le principe 'ne pas arreter la detection si un extrait echoue' est pertinent. L'implementation doit cependant rendre l'echec observable et ne pas annoncer un exemplaire present quand le fichier n'existe pas. La meme logique vaut pour un flush impossible : une conservation en memoire est preferée a une perte, mais l'orchestrateur doit connaitre l'etat degrade.

Correction ciblée : retourner un resultat explicite (succes, erreur, chemin) pour l'exemplaire; poser `FLAG_EXEMPLAR` seulement apres succes; ajouter compteur d'echecs et journal structure. Faire retourner/lever un statut de flush exploitable et exposer le nombre de tentatives dans `health/stats`.

A-03 - Separation presentation, transport et commandes

Le HTML embarque simplifie le deploiement, mais 400 lignes de presentation dans `viz.py` masquent la logique HTTP. Extraire le dashboard dans un asset Python dedie ou un fichier statique embarque, garder le handler sur les routes et introduire des fonctions de validation/serialisation. Dans la CLI, deleguer chaque commande a un module de service ou au moins a des helpers testes.

A-04 - Evolution de l'index de clusters

La recherche lineaire est acceptable au demarrage et a faible cardinalite; elle devient une dette de performance avec de nombreux clusters. Un index par `bin_peak` ou par `cle` (`bin`, `canal`, `classe de delai`) permettrait de limiter les candidats avant `fingerprints_match`, en preservant la regle actuelle et la simplicite de SQLite.

5. Robustesse, testabilite et maintenabilite

Robustesse runtime

- EventStore est globalement solide : RLock sur le buffer, transactions et lots conserves en cas d'echec. En revanche, get_events/get_stats appellent flush avant lecture; une requete dashboard peut donc faire du travail d'ecriture et attendre le verrou. Separer la visibilite des evenements ecrits et le flush force reduirait le couplage lecture/ecriture.
- BruitTrackHandler utilise store et config comme attributs de classe, affectes dans run_viz_server. C'est compact mais empeche d'heberger proprement deux serveurs independants dans le meme processus et rend les tests dependants de sous-classes. Une factory de handler fermee sur ses dependances est plus nette.
- Engine arrete toujours capture et store, y compris quand ils sont injectes de l'exterieur. Documenter l'ownership ou fournir close_resources evitera les surprises chez les integrations/test doubles.

Testabilite

- La suite est rapide et couvre les fonctions essentielles, notamment la simulation DSP, SQLite en memoire et les endpoints HTTP. C'est un excellent choix pour une cible sans materiel en CI.
- Il manque des tests de chemin nominal Engine.start, d'echecs disque d'exemplaire, de propagations d'echec de flush, de concurrence dashboard/capture et de performance de ClusterIndex avec plusieurs milliers de clusters.
- Le workflow CI n'execute pas ruff format --check, bien que cette commande fasse partie des commandes de projet. Elle doit devenir une gate pour eviter que le depot passe avec du formatage non conforme.

Maintenabilite

- Les dataclasses Config et SoundEvent constituent de bons contrats. Etendre ce modele aux DTO API et aux resultats de persistance reduira Any et les clefs magiques.
- Des constantes sont parfois bien nommees (budgets, seuils capture), mais d'autres policies sont directement integrees a de longues methodes. Extraire les politiques de transition de EventDetector facilitera les changements et les tests de table de decision.

6. Plan de refactorisation pragmatique

Priorite	Action	Gain	Critere de fin
P0	Unifier le chemin de lecture Engine.start/step et tester la sante capture.	Retablit la surveillance annoncee en production.	Test reproduit trois blocs lents et confirme une alerte/metric.
P0	Rendre les echecs d'exemplaire et de flush explicites.	Etat DB/fichiers fiable, diagnostic possible sans crash.	Aucun flag exemplaire sans fichier; echec flush visible dans health.
P1	Extraire la presentation dashboard et une factory Handler configuree.	viz.py devient testable par route et plus simple a modifier.	Handler HTTP sous 200 lignes hors assets; deux instances isolees testables.
P1	Introduire Protocol Capture, DTO EventView et resultats types.	Contrats explicites et moins de Any/dicts fragiles.	Mypy ou pyright optionnel sans dette de type nouvelle.
P2	Indexer ClusterIndex par caracteristiques grossieres.	Temps de detection stable avec l'historique.	Benchmark a 10 000 clusters respecte le budget tick.
P2	Scinder __main__.py par commande et centraliser helpers.	CLI plus lisible, fonctions plus courtes et testables.	Chaque commande a un test direct et aucune fonction >60 lignes hors parseur.
P3	Ajouter ruff format --check a la CI, appliquer format et corriger encodage.	Hygiene continue et sorties lisibles.	Toutes les gates locales et CI sont vertes.

Decision architecture

Conserver l'architecture actuelle en couches : elle est proportionnee au materiel et ne justifie pas un framework ou une dependance supplementaire. Privilegier des refactorisations locales, des contracts types et une observabilite minimale. La priorite n'est pas de reorganiser pour organiser, mais de corriger A-01/A-02 et de rendre les frontieres runtime plus fiables.

Fichiers examines

src/bruittrack/{__main__,capture,config,dsp,events,legal,pipeline,store,viz}.py; tests/; pyproject.toml; .github/workflows/ci.yml; tools/{install_hp,module_check,check}. Les constats se rapportent a la revision citee en couverture.